Fundamentals of Data Structures

Prof. Pranjal Pandit

Unit 1 Introduction to Algorithms and Data Structures

- From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program).
- Data, Information, Knowledge and Data Structure;
- Abstract Data Types (ADT);
- Classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral).
- Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart).
- Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega),
- Step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.
- Algorithmic strategies: Divide and Conquer, Greedy.

Algorithm Analysis Techniques

Used to estimate how much time an algorithm takes to execute. They help us understand how an algorithm's running time grows as the input size (**n**) increases.

Techniques:

- Step-count method
- Analysis of Linear
- Quadratic
- Cubic and
- Logarithmic constructs

Step-Count Method

What is it?

The Step-Count Method is a technique **used to find the exact number of basic operations performed by an algorithm**.

A **step** is any simple operation such as:

- Assignment (=),
- Comparison (<, >, ==)
- Addition/Subtraction
- Multiplication
- Increment (i++)
- Printing
- Return statement

Instead of saying "this algorithm is fast," we count **how many steps** it performs.

Example 1: Find Maximum Element

```
Algorithm Max(A, n)
max = A[0]
for i = 1 to n-1
        if A[i] > max
            max = A[i]

return max
```

Statement	Number of Times
max = A[0]	1
i initialization	1
Loop comparison	n
Increment i	n−1
if comparison	n−1
Assignment inside if	at most n−1
return	1

Step Count

Total steps

$$T(n) = 1 + 1 + n + (n-1) + (n-1) + (n-1) + 1$$

Ignoring constants,

$$T(n) = 4n$$

Therefore,

Time Complexity = O(n)

Example 2: Sum of Array

```
sum = 0
for i = 0 to n-1
        sum = sum + A[i]
return sum
```

Operation	Count
sum = 0	1
initialization	1
comparison	n+1
increment	n
addition & assignment	n
return	1

Step Count

Total

$$T(n) = 3n + 4$$

Complexity

$$O(n)$$

Why Step Count?

It helps us derive

- Exact running time
- Time complexity
- Big-O notation

Sum of Even Numbers

```
Algorithm SumEven(A, n)

sum = 0
count = 0

for i = 0 to n-1
    if A[i] % 2 == 0
        sum = sum + A[i]
        count = count + 1

average = sum / count

return average
```

Find Maximum and Minimum Together

Algorithm MaxMin(A, n)

```
max = A[0]
min = A[0]

for i = 1 to n-1

    if A[i] > max
        max = A[i]

    if A[i] < min
        min = A[i]

return max, min
```

Count Duplicate Pairs

```
count = 0

for i = 0 to n-1

    for j = i+1 to n-1

        if A[i] == A[j]
            count++

return count
```

Matrix Multiplication (Classic)

```
for i = 0 to n-1

    for j = 0 to n-1

        sum = 0

        for k = 0 to n-1

            sum = sum + A[i][k] * B[k][j]

        C[i][j] = sum
```

Statement	Frequency
i = 0	1
i < n	n + 1
i++	n
j = 0	n
j < n	n(n + 1)
j++	n^2
sum = 0	n^2
k = 0	n^2
k < n	$n^2(n + 1)$
k++	n^3
sum = sum + A[i][k] * B[k][j]	n^3
C[i][j] = sum	n^2

Adding the frequencies:

$$T(n) = \quad 1 + (n+1) + n + n + n(n+1) + n^2 + n^2 + n^2 + n^2(n+1) + n^3 + n^3 + n^2$$

Simplifying,

$$\boxed{T(n) = 3n^3 + 6n^2 + 5n + 2}$$

Therefore,

$$\boxed{T(n) = O(n^3)}$$

Step-Count Practice Strategy

When solving any algorithm using the step-count method, follow these steps:

1.Write each executable statement separately (assignment, comparison, increment, arithmetic operation, return, etc.).
2.Determine how many times each statement executes:

1. Before a loop: usually once.
2. Inside a loop: multiply by the number of iterations.
3. Nested loops: multiply by the iterations of both loops.

3.Create a table like this:
4.Add all frequencies to obtain the exact step count $T(n)$.
5.Keep only the dominant term to determine the asymptotic time complexity (Big-O).

Statement	Frequency
`sum = 0`	1
`i = 0`	1
Loop comparison	$n + 1$
Loop increment	n
`sum = sum + A[i]`	n
`return sum`	1

2. Linear Construct

A **Linear Construct** means an operation repeats **n times**. Running time increases proportionally with input size.

Example

for i = 1 to n

  print(i)

If n = 5

Output: Loop runs **5 times**.

If n = 100→ Loop runs **100 times**.

If n doubles, running time approximately doubles.

Step Count

Each iteration performs one print.

Total steps

$$T(n) = n$$

Time Complexity

$$O(n)$$

3. Quadratic Construct

A **Quadratic Construct** occurs when one loop is inside another loop. The total number of iterations becomes

$$n \times n = n^2$$

Example

```
for i = 1 to n
    for j = 1 to n
        print(i,j)
```

Suppose

n = 3

Total Iterations: 9 iterations →3×3=9

4. Cubic Construct

A **Cubic Construct** occurs when there are **three nested loops**.

Example

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            print(i,j,k)
```

Suppose
n = 2
Iterations
Total
8 iterations

$$2 \times 2 \times 2 = 8$$

If
n = 100
Iterations
$100^3 = 1{,}000{,}000$

Step Count

$$T(n) = n^3$$

Complexity

$$O(n^3)$$

5. Logarithmic Construct

This is the most interesting one. Instead of increasing one by one, the problem size is reduced by **half** each time.

Example

```
while n > 1
    n = n / 2
```

Suppose
n = 16
Iterations: Only **4 iterations**.

Suppose
n = 1024

Only **10 iterations**.
Notice how a huge input requires only a few steps.

Step Count

The number of times we divide by 2 until reaching 1 is

$$\log_2 n$$

Therefore

$$T(n) = \log n$$

Complexity

$$O(\log n)$$

Comparison of the Four Constructs

Construct	**Typical Code**	**Number of Operations**	**Time Complexity**	**Example**
Linear	One loop	n	**O(n)**	Sum of array, attendance
Quadratic	Two nested loops	n^2	**$O(n^2)$**	Bubble Sort, comparing every pair
Cubic	Three nested loops	n^3	**$O(n^3)$**	Matrix multiplication (basic algorithm), processing 3D combinations
Logarithmic	Divide input by 2	log n	**O(log n)**	Binary Search

Visual Growth Comparison

n	Linear (n)	Quadratic (n^2)	Cubic (n^3)	Logarithmic ($\log_2 n$)
10	10	100	1,000	3.3
100	100	10,000	1,000,000	6.6
1,000	1,000	1,000,000	1,000,000,000	10
1,000,000	1,000,000	1,000,000,000,000	10^{18}	20

Key Observation

O(log n) grows the slowest and is the most efficient for large inputs.

O(n) grows proportionally with the input size.

O(n^2) becomes much slower as the input increases.

O(n^3) grows very quickly and is generally inefficient for large datasets.

Algorithmic strategies: Divide and Conquer, Greedy.